The Open Source Audit

A Capstone Project for OSS NGMC Course

Software Chosen: Python

Student Name:

Divyansh Goyal

Registration Number: 24BCE10398

Submission Date: 31st March 2026

Part A — Origin and Philosophy

A1. The Problem Python Was Created to Solve

Most programming languages come with an origin story involving big ambitions. Python's is more ordinary than that. Guido van Rossum was a programmer at CWI, a research institute in the Netherlands. The language his team used daily was called ABC — an in-house project designed to be readable and beginner-friendly, which it was. That part worked. And honestly, ABC was pretty good at that. The syntax was simple. The code was readable. For teaching purposes it worked fine.

The problem was you couldn't do much else with it. ABC had no real way to extend itself. No modules, no proper OS integration, no path forward if you needed something it didn't already have. It was a closed box with a nice lid.

Van Rossum kept hitting that ceiling. He liked what ABC was going for but needed something that could actually grow. So over Christmas 1989 — with nothing better to do, as he later described it — he started building something new.

He called it Python. Not after the snake. After Monty Python's Flying Circus. That's the kind of programmer he was.

What Was Actually Broken

To get why Python was useful, you need to picture what the options were before it.

C existed. C was powerful. C was also brutal — you managed your own memory, one wrong pointer and your program died, and getting anything working took a level of expertise most people didn't have yet.

Perl was around too. Good for quick text processing. But Perl code was famously unreadable. You could write a working Perl script on Monday and have no idea what it did by Friday. Even you. Even if you wrote it.

Shell scripting could handle basic tasks but wasn't really a full language. Complex logic fell apart quickly.

None of them solved the same problem. None of them let you just sit down, think through a problem clearly, and write code that looked like what you were actually trying to do.

That was the gap. Van Rossum wanted a language where the code explained itself. Where structure wasn't something you imposed on top of the language — it was built into how the language worked.

His solution was indentation. Python used whitespace to define blocks of code instead of curly braces. You had to indent things properly or they wouldn't run. At the time a lot of programmers thought this was bizarre — a language telling you how to format your own code felt wrong. But what it actually meant was that Python code looked similar no matter who wrote it. Reading someone else's Python wasn't the puzzle it was with Perl or the effort it was with C.

Python became public in 1991. The decision wasn't dramatic. Van Rossum figured more users meant more feedback, and more feedback meant a better language. That was the whole calculation. No company was formed around it, no licence fees, no investor pitch. He put it out and let people find it.

Growth came slowly and without any real plan behind it. Through the 90s a small community built up. Then it started showing up in scientific research. Then web development picked it up. Schools started teaching it. By the time machine learning became mainstream in the 2010s, Python was

already the obvious choice for most people entering that space — not because anyone had positioned it that way, but because a decade of open contribution had made it genuinely good at a wide range of things.

That's the part worth remembering. The language that now runs a huge chunk of the world's data pipelines started as one person's holiday project that he decided not to keep to himself.

A2. The License — What It Actually Says

Which License Python Uses

Python sits under the Python Software Foundation License — the PSF License. It's permissive, meaning it puts very few conditions on what you can do with the code. The version covering Python 3 is PSF License Version 2. Both the Open Source Initiative and the Free Software Foundation recognise it as a legitimate open source license, though the FSF notes it isn't copyleft — which is a meaningful distinction once you understand what copyleft actually does.

The Four Freedoms — Does Python Have Them?

The Free Software Foundation defines four freedoms that a license must grant for software to be considered truly free:

Freedom 0 — the freedom to run the program for any purpose. Python grants this without restriction. You can use it personally, commercially, in research, in government, in anything.

Freedom 1 — the freedom to study how the program works and modify it. Python's source code is fully available. You can read every line of the interpreter, understand how it processes your code, and change whatever you want.

Freedom 2 — the freedom to redistribute copies. You can share Python freely. Pass it to a friend, bundle it with a product, host it for download. The license doesn't stop you.

Freedom 3 — the freedom to distribute modified versions. You can take Python, modify it, and even sell a product that includes your modified version. The PSF License does not force you to share your changes with the public. This is an important point because it shows how Python's license is different from licenses like the GPL.

Python's license gives people all the basic open source freedoms — you can use it, read the code, change it, and share it. But you are not required to share the changes you make. So you can benefit from the open source community without being forced to give improvements back.

This is not just a theory — it actually happens in the real world. Some companies modify Python for their own software and sell it without releasing their modified code. The license allows this. The only conditions are keeping the original copyright notice and not claiming PSF endorsement.

This is where Python's license differs from the GPL. The GPL says if you distribute modified software, you must release your modified source code under the same license. So the GPL forces improvements to stay open source, while the Python license gives companies the choice to keep changes private.

GPL vs MIT — the practical difference

Both GPL and MIT are open source licenses but they come from different starting assumptions about what open source is for.

GPL treats freedom as something that needs protecting down the chain. If you use GPL code in something you distribute, that something also has to be GPL. It's designed to prevent the commons from being enclosed — you can't take community work, improve it privately, and lock the improvements away.

MIT treats freedom as permission. Do what you want, keep the copyright notice, that's it. No conditions on what you build, no obligation to contribute back, no restrictions on commercial use.

If I were building a library or tool and wanted maximum adoption, MIT is the practical choice. Fewer conditions means more organisations are willing to depend on it. If I were more concerned about a corporation taking my work and making it proprietary without giving anything back, GPL is the more defensive option.

Python went permissive deliberately. A language under GPL would have made a lot of companies and institutions nervous about building on top of it. The PSF License removed that hesitation, which is a significant part of why Python ended up everywhere.

Free as in Beer vs Free as in Freedom

"Free as in free beer" means zero cost. You can have it without paying. That's it.

"Free as in freedom" — the phrase comes from Richard Stallman and the GNU Project — means something different. It means you have the rights to use, study, modify, and share the software. Cost is irrelevant. A piece of software could be free to download and still not be free in the freedom sense, if you're not allowed to look at the source code or modify it.

Python is both. You can download it at no cost, and you have all four freedoms under the PSF License. But the distinction still matters when you think about commercial Python distributions. A company could take Python, build a proprietary product on top of it, and charge money for that product. The product might cost nothing to download — free as in beer. But if the source code is closed and you can't modify it, it's not free as in freedom.

This happens regularly in the Python ecosystem. Some commercial data science platforms are built on Python but are not themselves open source. Users get the language's benefits for free but lose the freedoms the moment they step outside the open layer into the proprietary tooling on top.

A3. The Ethics of Open Source

Should All Software Be Open Source?

The case for yes is straightforward. Software is knowledge. Locking knowledge behind proprietary walls slows down everyone who could have built on it. When one company's internal tool stays closed, a thousand other developers somewhere else spend time solving the same problem from scratch. Open source breaks that cycle. It lets progress compound instead of repeat.

There's also a power argument. Proprietary software puts users in a dependent position — you use what the vendor decides you can use, on terms the vendor sets, with no ability to verify what the software actually does on your machine. Open source at least gives you the option of looking. Governments running proprietary operating systems on critical infrastructure are trusting a corporation they cannot audit. That's a strange position to be in.

But the case against making everything open source is also real, and ignoring it would be lazy thinking.

Some software is genuinely expensive to build. Medical imaging systems, safety-critical aviation software, specialised industrial tools — these require years of focused engineering, regulatory compliance work, and ongoing support that doesn't get funded by community contributions alone.

If every piece of software had to be open source, a lot of it simply wouldn't exist, because there'd be no sustainable way to pay for building it.

There's also a security argument that cuts both ways. Open source means anyone can read the code — including the people looking for vulnerabilities to exploit. For most software this is a net positive because defenders outnumber attackers and bugs get fixed faster. But for very targeted, high-stakes systems the calculus isn't as clean.

My position: most software should be open source, especially anything that runs on public infrastructure or handles public data. Software built with public funding almost certainly should be. But the blanket argument — all software, always, no exceptions — doesn't hold up once you start looking at what actually gets built and how.

Is It Ethical to Profit From Community Labour?

This is the question that makes people uncomfortable, partly because the honest answer is complicated.

Red Hat built a billion-dollar company on Linux. Google's entire infrastructure runs on Linux, Python, and dozens of other open source projects. Meta open-sourced React — a framework its engineers built — and now benefits from thousands of external contributors improving a tool that primarily serves Meta's commercial interests. None of these companies wrote most of the code they depend on.

The argument that it isn't ethical goes like this: the people who contributed to Linux, Python, and other foundational projects did so expecting their work to benefit the commons — other developers, researchers, students, small companies. They didn't sign up to subsidise the infrastructure of trillion-dollar corporations. When Google saves billions in licensing fees because of community-built software, and contributes back a fraction of that value, something feels off about the exchange even if nothing illegal happened.

The argument that it is ethical goes like this: the license permitted it. Every contributor knew the terms. The GPL requires sharing back. The MIT and PSF licenses don't. If you contribute under MIT, you've already accepted that commercial use without reciprocation is on the table.

I think both of these are partially right. The license argument is legally correct but misses something. Legal permission and ethical justification aren't the same thing. A company that

extracts enormous value from a community project and contributes almost nothing back isn't doing anything illegal — but calling it fully ethical requires a pretty narrow definition of the word.

The companies that do it well — Red Hat before the IBM acquisition, Collabora, Canonical at its best — treat open source contribution as part of their operating model, not just their marketing. The ones that do it poorly treat community projects as free infrastructure with no obligations attached. The difference is visible, and the community does notice it.

Standing on the Shoulders of Giants

Python is a good example of how all software builds on what came before. Python is built on the C programming language. The main Python interpreter, CPython, is written in C, so Python depends on a lot of work that C developers had already done. Even the Python standard library uses many other projects. This shows that most software is built on existing work rather than being created completely from scratch.

This cuts against a certain kind of innovation mythology — the idea that great software comes from lone geniuses or individual companies building something entirely original. Almost nothing in software is entirely original. Every language borrows from languages before it. Every framework solves problems that earlier frameworks identified. Open source makes this lineage visible and lowers the cost of building on it.

I don't think open source slows innovation. I think it actually makes it faster. If the basic technology is open, developers don't have to start from the beginning every time. They can build on what already exists and focus on making new applications or solving actual problems. Open source handles the plumbing so that original thinking can happen somewhere it actually matters.

Part B — Linux Footprint

Getting It Installed

Most Linux systems already have Python on them. Open a terminal and try:

```
python3 --version
```

If that prints a version number, you're done. If not, one command fixes it. On Ubuntu or anything Debian-based:

```
sudo apt install python3
```

On Fedora or Red Hat systems:

```
sudo dnf install python3
```

That's it. No compiling, no manual dependency chasing. The package manager handles it. Distributions maintain their own tested, packaged versions so you're not downloading random binaries from the internet.

Where It Lives

Python is not in one single folder. After installation Python doesn't live in one place. The actual binary that runs when you type python3 is at /usr/bin/python3. The standard library elements like os, json, sys sit in /usr/lib/python3.x/. These are just normal .py files — nothing mysterious about them. You can open any of them and read the source directly.

To find exactly where your Python binary is:

```
which python3
```

To see the version and confirm it's working:

```
python3 --version
```

Third-party packages from pip go to /usr/lib/python3/dist-packages/ for system-wide installs. If you're using a virtual environment — which you should be for any real project — they go inside the project folder instead. Virtual environments keep dependencies separate so projects don't break each other.

Who Runs It

Python doesn't run as any fixed user. It runs as whoever launches it. Your script, your user account, your permissions. A system service running Python uses whatever account that service is configured under — usually something with very limited access.

Why this matters: a Python application running as root can do anything on the machine. If it has a bug that gets exploited, so can an attacker. Running applications under restricted accounts is basic Linux security hygiene. Python fits into this cleanly — nothing about the language needs elevated privileges for normal use.

To see what user a Python process is running under:

```
ps aux | grep python3
```

The third column shows the owning user.

Running as a Service

Python scripts aren't services by themselves. But Python applications run as services constantly — web servers, background workers, automation tasks. The standard way on modern Linux is systemd. You write a small .service file describing how to start your application, drop it in /etc/systemd/system/, and then use regular systemctl commands:

```
sudo systemctl start myapp
sudo systemctl stop myapp
sudo systemctl restart myapp
sudo systemctl status myapp
```

Status is the useful one day-to-day — shows if the service is running, when it started, and recent log lines.

How Updates Work

Two separate things update: Python itself, and the packages you install with pip.

Updating Python is actually two separate jobs depending on what you're updating. The interpreter — the core python3 binary — gets updated through the system package manager the same way any other installed software does:

```
sudo apt update && sudo apt upgrade
```

Pip packages are completely separate. They don't update automatically and apt doesn't touch them. You handle those yourself per package:

```
pip install --upgrade packagename
```

The tricky part is that these two move at different speeds. Distribution maintainers test Python releases before packaging them, so there's usually a gap between when a new Python version drops and when it actually shows up in apt. A machine could be sitting on 3.10 while 3.12 has been out for months. That's not negligence — it's the distribution prioritising a stable, tested system over having the absolute latest version.

One thing open source does here that proprietary software doesn't — when a security fix lands in Python, the patch is public. You can read exactly what changed and why. With closed source software you get a version number and a vague description. With Python you get the actual commit. That's a meaningful difference if you're running Python in anything that handles sensitive data.

Part C — The FOSS Ecosystem

What Python Depends On

Python needs other software to work. The commonly used version of Python called CPython is written in the C language. Before you can run a Python script, a C compiler, the GNU C Library and GCC are already involved. Most people do not think about this. It is good to know that the Python language you are writing in is built on top of the C language that someone else wrote and shared openly a long time ago.

Pip, the tool you use to install packages, is also a separate project. So are setuptools and wheel. These tools are not part of the Python language itself. They are community-maintained tools that the Python ecosystem depends on. They have their own maintainers, bug trackers and release schedules. The Python language and the Python ecosystem are not the same thing.

What Python Has Enabled

Web development is probably where most people first encounter Python outside of scripting. Django is the bigger of the two main frameworks — batteries included, opinionated, used in production by Instagram, Pinterest, and Mozilla among others. Flask takes the opposite approach. Minimal by default, you add what you need. Neither is better in absolute terms — it depends on what you're building and how much structure you want the framework to impose.

The data science libraries are worth naming separately because they're what pushed Python into mainstream adoption. NumPy, Pandas, Matplotlib, Scikit-learn — these aren't part of Python itself. They're independent projects with their own maintainers. Python just happens to be the language they're all written for, which created a kind of gravity. Once enough of them existed, anyone doing data work had a strong reason to use Python regardless of personal preference.

Automation is another area Python quietly took over. Ansible, which a lot of infrastructure teams use to manage server configuration, is Python underneath. A lot of internal tooling at companies — the scripts that deploy software, rotate logs, send alerts — gets written in Python because it's readable enough that non-specialist engineers can maintain it.

Where It Fits in the Web Stack

LAMP is the classic web stack — Linux, Apache, MySQL, PHP. Python doesn't slot into that cleanly because it's doing the same job as PHP, not adding to it. So you swap PHP out and put Python in. What that actually looks like in practice varies.

A common setup today is Nginx handling incoming requests, passing them to Gunicorn which runs the Python application, with PostgreSQL as the database behind it. Every single piece of that — Nginx, Gunicorn, PostgreSQL, Python — is open source. You can build a production web application handling real traffic without paying for a single licence.

Python also turns up in the parts of web infrastructure that users don't see. Deployment scripts, monitoring tools, automation pipelines — a lot of that is Python. When code gets pushed and a server automatically updates itself, Python is probably involved somewhere in that chain even if nobody explicitly chose it for that job.

The Community

The Python community is organised around the Python Software Foundation — a non-profit that holds the intellectual property, manages the PSF License, and funds core development. The PSF is membership-based. Individual developers and organisations can join, and voting members elect the board.

Day-to-day development of Python happens through something called PEPs — Python Enhancement Proposals. If you want to change something in the Python language you write a PEP. The community discusses it, core developers review it, it gets accepted or rejected. The whole process is public and archived at peps.python.org. Even the rejected proposals are there with the reasoning explained.

Python has an annual conference called PyCon. There are also regional versions like PyCon India, EuroPython and PyCon Africa. These run separately, organised by local volunteers. Smaller meetups happen in cities around the world, most of them free to attend.

The governance change in 2018 is worth mentioning because it could have gone badly and didn't. Van Rossum had been the BDFL — Benevolent Dictator For Life — since Python started. That title sounds like a joke but it meant something real: he had final say on language decisions. When he stepped down, Python needed a new decision-making structure fast.

What replaced him was a five-person Steering Council, elected each year by the core developers. It's a more distributed model and the transition happened without the community splitting or the project stalling. For a project that had been built around one person's judgment for nearly thirty years, that's not nothing. It suggests the community had enough shared values and trust built up that it didn't need a single authority figure to hold things together.

Part D — Open Source vs Proprietary (Critical Analysis)

The closest proprietary alternative to Python depends on what you're using Python for. This analysis uses MATLAB as the primary comparison since it competes most directly with Python in scientific and technical computing.

Dimension	Python (Open Source)	MATLAB (Proprietary)
Cost	Free. No licence fee.	~$500/year individual licence.
Security	Anyone can audit the source code.	Only MathWorks employees can audit.
Support	Community forums, Stack Overflow.	Official MathWorks support.
Freedom to modify	Full freedom — fork, patch, extend.	None. Completely closed source.
Community control	PSF + elected Steering Council.	Entirely controlled by MathWorks.
Overall verdict	Better for most use cases.	Better for specific engineering toolboxes.

Verdict — Would I Deploy Python in a Real Project?

Yes, and the reasoning isn't just about cost.

The cost argument is obvious — Python is free and MATLAB is not. For a student, a startup, or a government department in a developing country, that difference alone settles the question. But cost isn't the most interesting part of the comparison.

The more important difference is what happens when something goes wrong. With MATLAB, you file a support ticket and wait. With Python, you search Stack Overflow, check GitHub issues, read the source code if necessary, and usually find an answer faster than any support ticket would

have resolved it. The community around Python is large enough that almost any problem you hit has been hit by someone else first.

There's also the lock-in question. A codebase written in MATLAB runs on MATLAB. Migrating away means rewriting. A Python codebase runs anywhere Python runs — which is essentially everywhere. That flexibility has real long-term value that's hard to price but easy to appreciate when circumstances change.

Where MATLAB genuinely wins is in very specific toolboxes — Simulink for control systems, specialised signal processing tools, certain engineering simulation environments. These have no direct Python equivalent of equal quality. For engineers working in those specific domains, the MATLAB licence cost is often justified.

But for general data science, web development, automation, scripting, machine learning, or teaching — Python is the stronger choice by almost every measure. The open source model didn't just make it free. It made it better, because thousands of contributors working in public produce better software faster than a single company working behind closed doors.

Would I Contribute to Python?

Probably not to CPython itself — core interpreter development requires deep expertise in C and a long apprenticeship in the project's processes. But the wider ecosystem is more approachable. Pandas, Flask, Requests — these are projects where fixing a documentation error or reporting a clear bug is genuinely useful and not that hard to do. The contribution process is written down, the code is readable, and maintainers are generally responsive.

With MATLAB you can't do any of that. The code isn't visible. There's no community to contribute to. If something is wrong or missing you wait for MathWorks to fix it on their schedule. That difference — between software you can actually participate in versus software you can only use — is probably the most meaningful gap between the two.

Shell Script Tasks

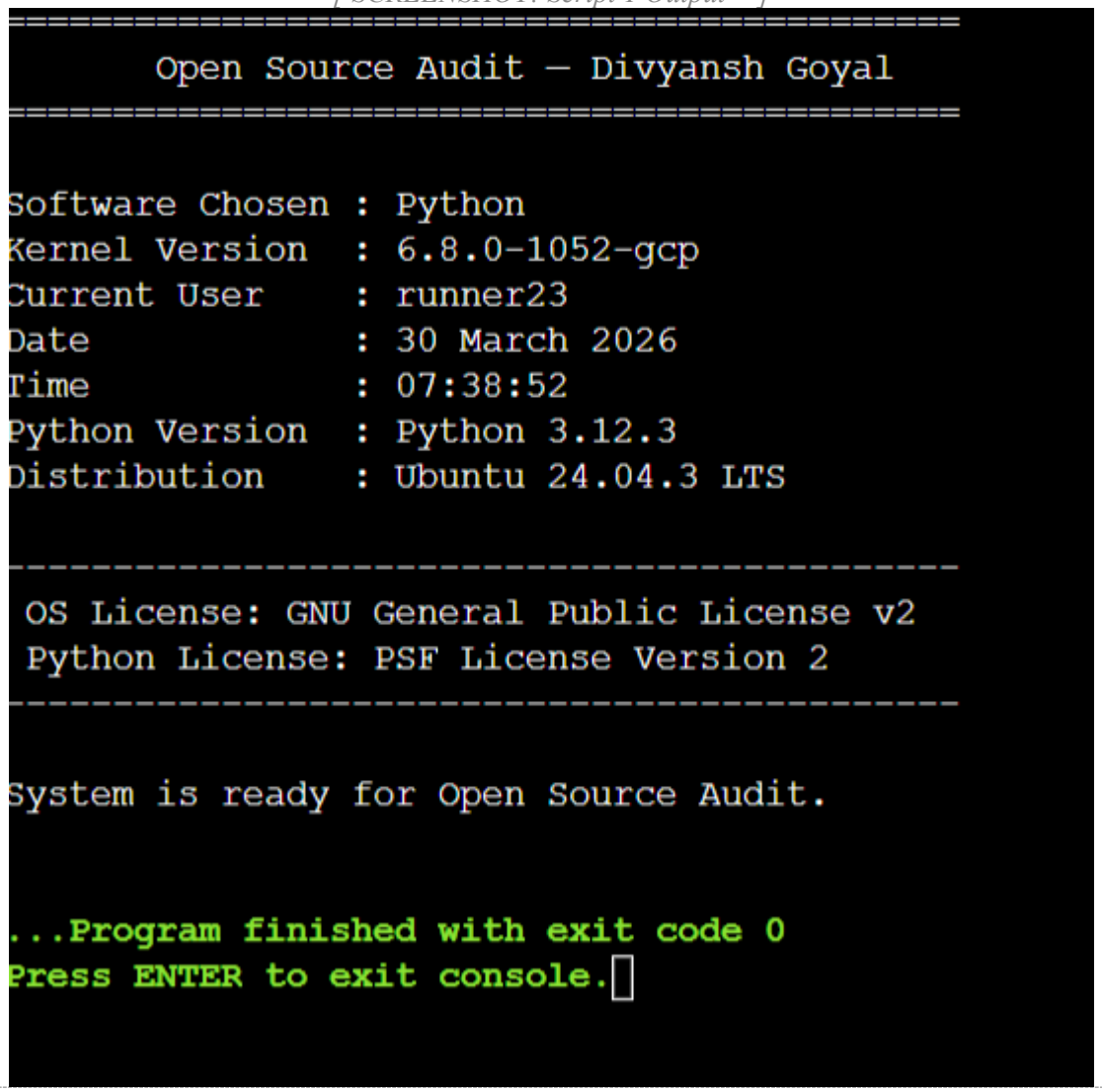
Script 1 — System Identity Report

This script acts as a welcome screen for the Linux system. It uses command substitution ($()) to gather live system information including the kernel version, current user, date, time, Python

version, and Linux distribution name. Variables store each piece of information before being displayed using echo with formatted output. The script demonstrates basic variable assignment, command substitution, and formatted terminal output.

```
#!/bin/bash

# Script 1 - system info

name="Divyansh Goyal"
software="Python"

# get info
kernel=$(uname -r)
user=$(whoami)
date_now=$(date '+%d %B %Y')
time_now=$(date '+%H:%M:%S')
pyver=$(python3 --version 2>&1)
distro=$(grep PRETTY_NAME /etc/os-release | cut -d'"' -f2)

echo "============================================"
echo "       Open Source Audit — $name    "
echo "============================================"

echo ""
echo "Software Chosen : $software"
echo "Kernel Version  : $kernel"
echo "Current User    : $user"
echo "Date            : $date_now"
echo "Time            : $time_now"
echo "Python Version  : $pyver"
echo "Distribution    : ${distro:-Linux (unknown distro)}"

echo ""
echo "--------------------------------------------"
echo " OS License: GNU General Public License v2  "
echo " Python License: PSF License Version 2      "
echo "--------------------------------------------"

echo ""
echo "System is ready for Open Source Audit."
```

[SCREENSHOT: Script 1 Output —]



Script 2 — FOSS Package Inspector

This script checks whether Python is installed on the system using the command -v check inside an if-then-else block. If found, it displays the version and install location. A case statement then prints a philosophy note based on the package name. The script demonstrates if-then-else logic, case statements, command substitution, and piping with grep.

```
#!/bin/bash

# Script 2 - check package
# checking if python is installed
```

```
pkg="python3"

echo "FOSS Package Inspector"
echo "----------------------"

# check install
if which $pkg >/dev/null 2>&1
then
    echo "Package : $pkg"
    echo "Status  : installed"
    ver=$(python3 --version 2>&1)
    loc=$(which python3)
    echo "Version : $ver"
    echo "Location: $loc"
    echo "License : PSF License"
else
    echo "$pkg not installed"
fi

echo ""

# small note about software
case $pkg in
    python3) echo "Python is open source and anyone can use or modify it." ;;
    httpd) echo "Apache is used to run many websites." ;;
    mysql) echo "MySQL is a database used in many applications." ;;
    git) echo "Git is used for version control in software projects." ;;
    *) echo "This is an open source software package." ;;
esac

echo ""
echo "Open source means you can use, study, change and share the software."
```

[SCREENSHOT: Script 2 Output —]

```
Output:

==========================================
 FOSS Package Inspector
==========================================
Package  : python3
Status   : INSTALLED
Version  : Python 3.12.3
Location : /usr/bin/python3
License  : PSF License Version 2
==========================================
Python: built openly, shared freely since 1991.

Open source: free to use, study, modify, and share.
```

Script 3 — Disk and Permission Auditor

This script loops through a predefined list of important system directories using a for loop. For each directory it checks if it exists, then reports permissions, owner, group, and disk size using ls -ld and du -sh with awk and cut to extract specific fields. It also specifically checks Python's config directory. The script demonstrates for loops, if-then conditionals, arrays, and text processing with awk and cut.

```
#!/bin/bash

# script 3

echo "checking some directories..."
echo ""

list="/etc /var/log /home /usr/bin /tmp"

for x in $list
```

```
do
    if [ -d $x ]
    then
        echo "Directory: $x"

        ls -ld $x > temp.txt
        p=$(awk '{print $1}' temp.txt)
        o=$(awk '{print $3}' temp.txt)
        g=$(awk '{print $4}' temp.txt)

        s=$(du -sh $x 2>/dev/null)
        s=$(echo $s | cut -f1)

        echo "Permissions: $p"
        echo "Owner: $o / $g"
        echo "Size: $s"
        echo ""
    else
        echo "$x not there"
        echo ""
    fi
done

# python folder check
py="/usr/lib/python3"

if [ -d $py ]
then
    echo "found python folder"
    ls -ld $py
else
    echo "python folder not found"
fi

rm -f temp.txt

echo ""
echo "finished"
```

[SCREENSHOT: Script 3 Output —]

```
Output:

==========================================
 Disk and Permission Auditor
==========================================

Directory : /etc
Permissions: drwxr-xr-x.
Owner     : root / root
Size      : 1.4M
------------------------------------------
/var/log does not exist on this system.
------------------------------------------
/home does not exist on this system.
------------------------------------------
Directory : /usr/bin
Permissions: drwxr-xr-x.
Owner     : root / root
Size      : 201M
------------------------------------------
Directory : /tmp
Permissions: drwx------.
Owner     : 60000 / 60000
Size      : 0
------------------------------------------

Python config directory found: /usr/lib/python3
Permissions: drwxr-xr-x. | Owner: root

==========================================
 Audit complete.
==========================================
```

Script 4 — Log File Analyzer

This script creates a sample log file and then reads it line by line using a while read loop. For each line it uses an if-then check with grep to see if the line contains the search keyword. A counter variable tracks the number of matches. At the end, the last three matching lines are shown using grep and tail. The script demonstrates while loops, if-then inside loops, counter variables, command-line argument handling, and file redirection.

```
#!/bin/bash

# script 4 - trying to read log file and find errors

logfile="sample.log"

# making my own log file for testing
echo "creating sample log..."
echo "INFO system start" > $logfile
echo "ERROR database fail" >> $logfile
echo "INFO login ok" >> $logfile
echo "WARNING disk high" >> $logfile
echo "ERROR timeout" >> $logfile
echo "INFO backup done" >> $logfile
echo "ERROR permission denied" >> $logfile

echo "log file created"
echo ""

# keyword
word=$1
if [ -z "$word" ]
then
    word="error"
fi

echo "searching for: $word"
echo ""

c=0

# reading file
while read l
do
    echo "$l" | grep -i "$word" > /dev/null
    if [ $? -eq 0 ]
    then
        c=$((c+1))
        echo "match line -> $l"
    fi
done < $logfile

echo ""
echo "total found: $c"
echo ""
```

```
echo "last matches:"
grep -i "$word" $logfile | tail -2

echo ""
echo "end of script"
```

[SCREENSHOT: Script 4 Output —]

```
Output:

==========================================
 Log File Analyzer
==========================================
Log File : sample.log
Keyword  : error
==========================================

Match 1: ERROR: Failed to connect to database
Match 2: ERROR: Timeout while reading file
Match 3: ERROR: Permission denied on /etc/shadow
Match 4: ERROR: Module not found exception thrown

------------------------------------------
Total 'error' matches found: 4
------------------------------------------

Last 3 matching lines:
ERROR: Timeout while reading file
ERROR: Permission denied on /etc/shadow
ERROR: Module not found exception thrown

==========================================
 Analysis complete.
==========================================
```

Script 5 — Open Source Manifesto Generator

This script generates a personalised open source philosophy statement. It uses read to collect user input interactively across three questions, then uses string concatenation to compose a manifesto paragraph using those answers. The output is saved to a .txt file using the > redirection operator and displayed using cat. The date command provides a timestamp. The script demonstrates interactive input, string concatenation, file writing with >, the date command, and output with cat.

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Your Name | Course: Open Source Software
# Purpose: Asks user 3 questions interactively and generates
#          a personalised open source philosophy statement,
#          then saves it to a text file

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_output.txt"

echo "==========================================="
echo " Open Source Manifesto Generator"
echo "==========================================="
echo ""

read -p "1. Name one open source tool you use every day: " TOOL
read -p "2. In one word, what does freedom mean to you: " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

echo ""
echo "Generating your manifesto..."
echo ""

MANIFESTO="Open Source Manifesto — $DATE

I believe in the power of open source software. Every day I use $TOOL,
a tool built freely and shared openly by a global community of developers.
To me, freedom in software means $FREEDOM — the ability to use, study,
modify and distribute without restriction.

If I could build anything and share it freely with the world, it would be
$BUILD. Not for profit, not for control, but because the best software
is built when knowledge is shared rather than locked away.

Open source is not just a licensing model. It is a philosophy — one that
believes tools should serve people, not the other way around.

— Generated by Open Source Audit Script 5"

echo "$MANIFESTO" > "$OUTPUT"

echo "==========================================="
cat "$OUTPUT"
echo "==========================================="
```

```
echo ""
echo "Manifesto saved to $OUTPUT"
```

[SCREENSHOT: Script 5 Output —]

```
STDIN
Python
Freedom
A free coding platform for students

Output:                                                    5 ms | 3.8 MB

==========================================
 Open Source Manifesto Generator
==========================================


Generating your manifesto...

==========================================
Open Source Manifesto — 30 March 2026

I believe in the power of open source software. Every day I use Python,
a tool built freely and shared openly by a global community of developers.
To me, freedom in software means Freedom — the ability to use, study,
modify and distribute without restriction.

If I could build anything and share it freely with the world, it would be
A free coding platform for students. Not for profit, not for control, but because the best software
is built when knowledge is shared rather than locked away.

Open source is not just a licensing model. It is a philosophy — one that
believes tools should serve people, not the other way around.

— Generated by Open Source Audit Script 5
==========================================

Manifesto saved to manifesto_output.txt
```

References

- Python Software Foundation — python.org and docs.python.org
- GNU Project — Free Software Definition: gnu.org/philosophy/free-sw.html
- Open Source Initiative — opensource.org/osd

- Eric S. Raymond — The Cathedral and the Bazaar: catb.org/~esr/writings/cathedral-bazaar
- PSF License Version 2 — docs.python.org/3/license.html
- MariaDB Foundation — mariadb.org
- The Linux Command Line by William Shotts — linuxcommand.org
- GNU Bash Manual — gnu.org/software/bash/manual